KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 5 Quiz

Operatör Yükleme

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (10 puan)

4 • Değerlendirme Hedef 1

`geometri::Vektor2B` sınıfı skalerle çarpma için *üye* `operator*(int katsayi) const` tanımlar; böylece `v * 3` çalışır. Ancak `3 * v` yazımı bu üye fonksiyonla *tek başına* çalışmaz. Bu simetriyi sağlamak için hangisi DOĞRUDUR?

(a) Üye `operator*` zaten her iki yazımı da kapsar; `3 * v` sorunsuz derlenir, ek bir şey gerekmez.

(b) Sınıf dışında serbest bir `operator*(int katsayi, const Vektor2B& v)` tanımlanır; sol işlenen `int` olduğu için üye olamaz. Bu serbest fonksiyon yalnızca sınıfın public üye `operator*`'ına devrettiğinden `friend` OLMASINA gerek yoktur.

(c) `3 * v`'yi desteklemek için `int` tipine bir üye `operator*` eklemek gerekir; bu C++'ta olağan bir çözümdür.

(d) Serbest `operator*(int, const Vektor2B&)` yazılmalı ama `private` `bilesenler_`'e eriştiği için mutlaka `friend` bildirilmelidir.

Soru 2 (10 puan)

4 • Değerlendirme Hedef 4

`Vektor2B` için akış operatörü `operator<<(std::ostream& cikis, const Vektor2B& v)` bir üye fonksiyon olarak DEĞİL, serbest (ve genelde `friend`) bir fonksiyon olarak yazılır. Bunun temel nedeni nedir?

(a) Üye fonksiyonlar akışlarla hiçbir zaman çalışamaz; bu C++'ın genel bir kısıtıdır.

(b) `operator<<`'in sol işleneni `std::cout` gibi bir `std::ostream`'dir; üye yazılsaydı sol işlenen sınıfın kendisi (`this`) olmak zorunda kalırdı. Sol işleneni `std::ostream` olan bir üye `Vektor2B`'ye ekleyemezsiniz (ostream'i değiştiremezsiniz), bu yüzden serbest olmalıdır.

(c) Serbest yazmak yalnızca bir üslup tercihidir; üye olarak da tamamen aynı biçimde yazılabilirdi.

(d) `operator<<` üye olamaz çünkü `void` döndürmek zorundadır ve üye fonksiyonlar değer döndüremez.

Soru 3 (10 puan)

4 • Değerlendirme Hedef 2

`Vektor2B operator+(const Vektor2B& sag) const` bir referans (`Vektor2B&`) değil, *değer* (by value) döndürür. Buna karşılık `operator+=` `Vektor2B&` döndürür. Bu iki seçim için han-

Soru 3 (devamı)

gisi DOĞRUDUR?

(a) `operator+` da referans döndürmeli; değer döndürmek gereksiz bir kopya oluşturduğu için hatadır.

(b) `operator+` ikisinden de bağımsız YENİ bir nesne oluşturur; bu yereli referansla döndürmek fonksiyon bitince sarkan (dangling) referans üretirdi, bu yüzden değerle döndürür. `operator+=` ise var olan nesneyi (`*this`) yerinde güncellediği için ona referans döndürebilir.

(c) İkisi de değer döndürmelidir; `operator+=`'nın referans döndürmesi bir tasarım hatasıdır.

(d) Dönüş tipi tamamen keyfîdir; `operator+` ile `operator+=` arasında anlamlı bir fark yoktur.

Soru 4 (10 puan)

4 • Değerlendirme Hedef 4

`Vektor2B` indeksleme için iki aşırı yükleme tanımlar: `int& operator[](int indeks)` ve `int operator[](int indeks) const`. Bu const/non-const ikilisini tanımlamanın nedeni nedir?

(a) İkisi aynı işi yapar; biri gereksizdir ama derleyici her operatörün iki kez yazılmasını zorunlu kılar.

(b) const olmayan sürüm `int&` döndürerek değiştirmeye izin verir (`v[0] = 10;`); const sürüm ise salt-okunur bir `int` döndürür ve böylece const bir `Vektor2B` nesnesinin de `[]` ile okunabilmesini sağlar. Derleyici nesnenin const'luğuna göre uygun sürümü seçer.

(c) const sürüm yalnızca performans içindir; kaldırılsa da const nesneler yine `[]` ile okunabilirdi.

(d) non-const sürüm const nesnelerde, const sürüm ise const olmayan nesnelerde çağrılır.

Soru 5 (15 puan)

4 • Değerlendirme Hedef 2

`operator<<`, bir `Vektor2B`'yi `(x, y)` biçiminde yazdırır. Aşağıdaki `main` kesitini inceleyiniz:

```
13    Vektor2B v1(2, 3);
14    Vektor2B v2(4, 1);
15    std::cout << "v1=" << v1 << ", v2=" << v2 << '\n';
16    std::cout << "v1+v2=" << (v1 + v2) << '\n';
17    std::cout << "v1-v2=" << (v1 - v2) << '\n';
18    std::cout << "v1*3=" << (v1 * 3) << '\n';
19    std::cout << "3*v1=" << (3 * v1) << '\n';
20    v1 += v2;
21    std::cout << "v1 += v2 sonrasi v1=" << v1 << '\n';
```

Bu kesitin konsola bastığı çıktıyı satır satır yazınız.

Soru 6 (15 puan)

4 • Değerlendirme Hedef 3

`Vektor2B`, karşılaştırma için `bool operator==(const Vektor2B&) const = default;` ve `auto operator<=>(const Vektor2B&) const = default;` tanımlar; böylece altı karşılaştırma operatörü `bilesenler_` (bir `std::array<int,2>`) üye sırasına göre *sözlüksel* (lexicographic) türetilir. Aşağıdaki kesiti inceleyiniz:

Soru 6 (devamı)

```
   Vektor2B a(2, 3);
   Vektor2B b(2, 3);
   Vektor2B c(4, 1);
   std::cout << std::boolalpha;
   std::cout << "(2,3) == (2,3) -> " << (a == b) << '\n';
   std::cout << "(2,3) < (4,1) -> " << (a < c) << '\n';

   std::vector<Vektor2B> vektorler{Vektor2B(3, 1), Vektor2B(1, 5), Vektor2B(2, 2)};
   std::sort(vektorler.begin(), vektorler.end());
   std::cout << "siralanmis: ";
   for (const auto& v : vektorler) {
      std::cout << v << ' ';
   }
   std::cout << '\n';
```

Konsola basılan çıktıyı yazınız (`std::boolalpha` etkin olduğundan mantıksal değerler `true`/`false` biçiminde basılır).

Soru 7 (15 puan)

4 • Değerlendirme Hedef 2

`Vektor2B`'nin `private` alanı `std::array<int, 2> bilesenler_`; bileşik atama operatörünün bildirimi ise aşağıdaki gibidir:

```
Vektor2B& Vektor2B::operator+=(const Vektor2B& sag) {
   // GÖVDEYİ SİZ YAZIN
}
```

Bu fonksiyonun gövdesini yazınız. Nesneyi *yerinde* güncellemeli (iki bileşene `sag`'ın karşılık gelen bileşenlerini eklemeli) ve zincirlemeye izin verecek biçimde doğru değeri döndürmelidir.

Soru 8 (10 puan)

4 • Değerlendirme Hedef 4

`Vektor2B` için `operator<<`, sınıf içinde `friend std::ostream& operator<<(std::ostream&, const Vektor2B&)` biçiminde bildirilir. Bu operatörün neden *hem serbest* (*üye değil*) *hem de* `friend` olması gerektiğini, iki gerekçeyi ayrı ayrı belirterek açıklayınız.

Soru 9 (10 puan)

4 • Değerlendirme Hedef 3

C++20'de `auto operator<=>(...) const = default;` yazmanın avantajı nedir? Ayrıca, bir `Kesir` (pay/payda) tipini düşünün: `= default` karşılaştırma bu tip için neden *yanlış* sonuç verir ve böyle bir durumda ne yapmalısınız? `1/2` ile `1/3` örneği üzerinden açıklayınız.